ASSIGNMENT-11.5

NAME: SRAVANI ANUMALA

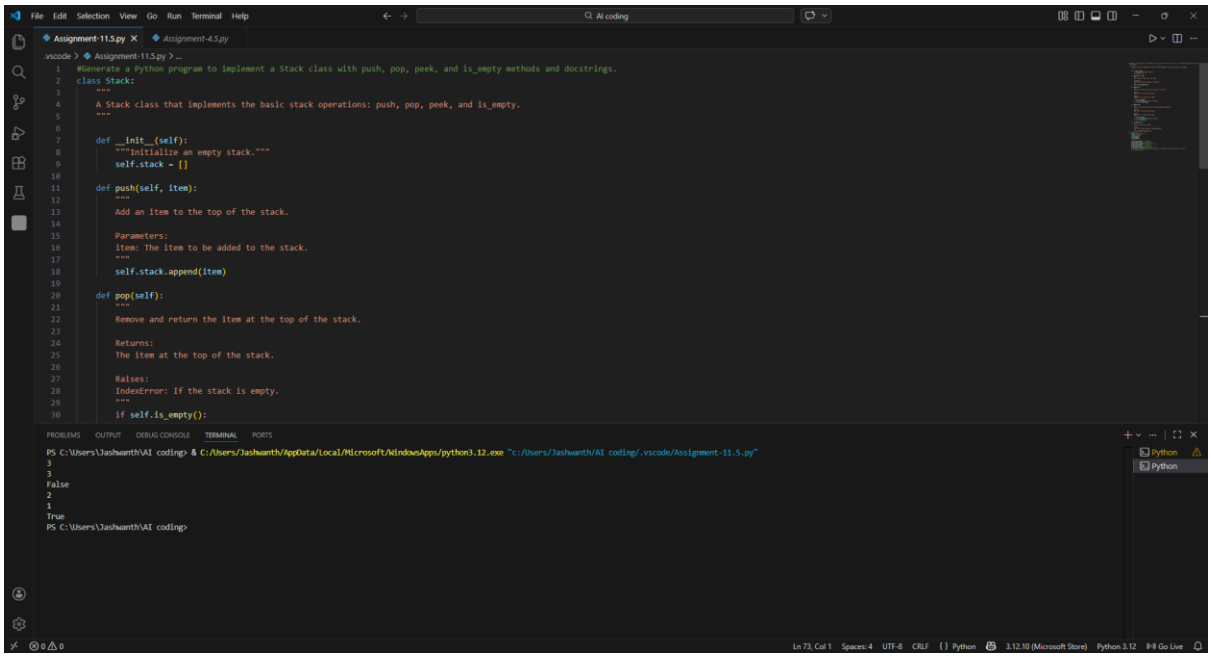
ROLLNO: 2303A510G7

BATCH: 30

TASK-1

PROMPT: Generate a Python program to implement a Stack class with push, pop, peek, and is_empty methods and show example usage.

CODE:



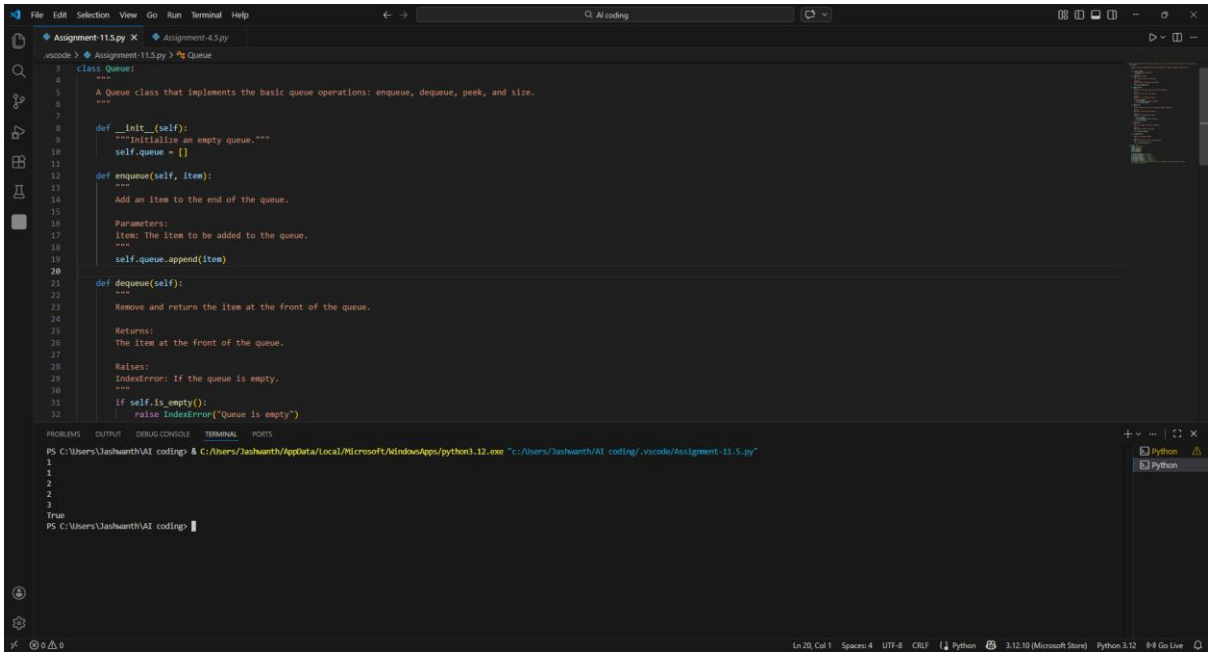
OBSERVATION:

The stack was implemented successfully using a class and Python list. The operations push, pop, peek, and is_empty worked correctly following the **LIFO (Last In First Out)** principle. The most recently added element was removed first when pop was executed.

TASK-2

PROMPT: Create a Python program to implement a Queue using a list with enqueue, dequeue, peek, and size operations.

CODE:



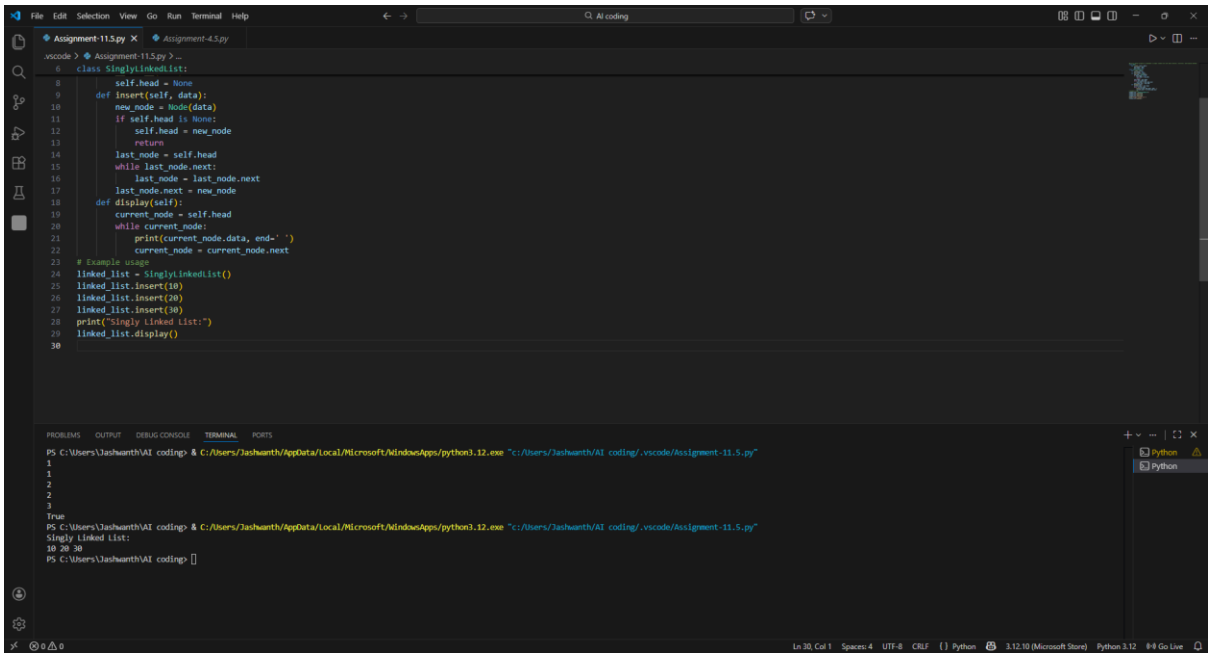
OBSERVATION:

The queue program worked correctly using a list to store elements. The operations enqueue, dequeue, peek, and size were implemented successfully. The queue followed the **FIFO (First In First Out)** principle where the first inserted element was removed first.

TASK-3

PROMPT: Write a Python program to implement a singly linked list with node creation, insertion, and display methods.

CODE:



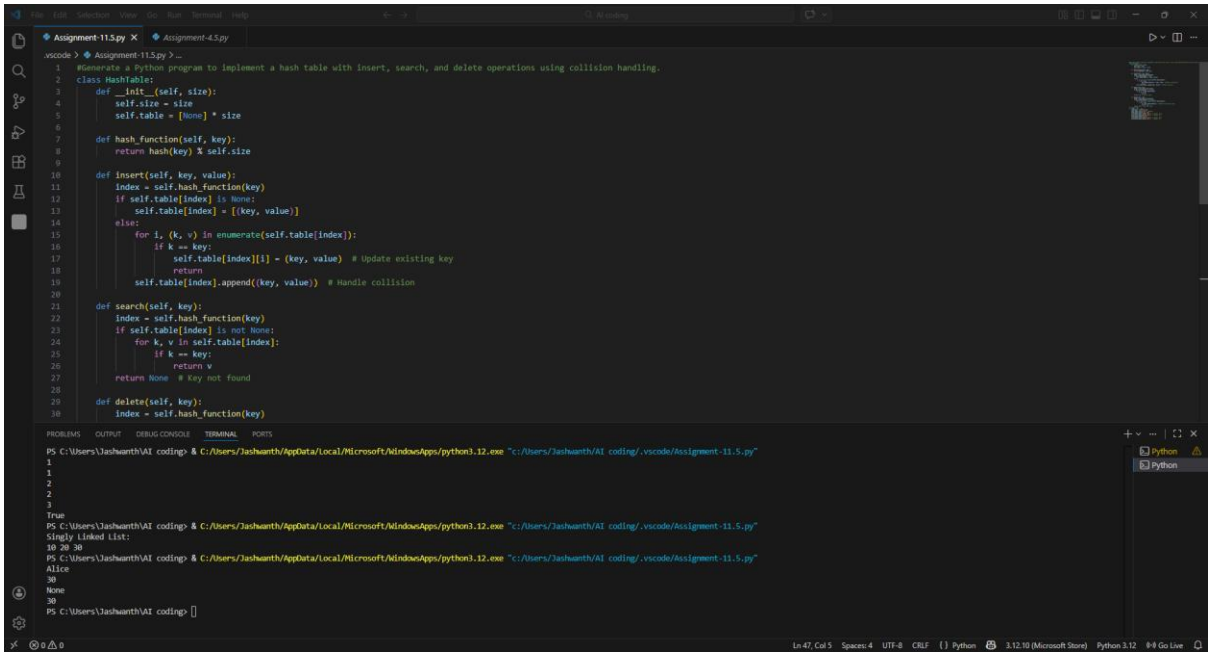
OBSERVATION:

The singly linked list was created using nodes connected through references. New nodes were inserted successfully, and the display function traversed the list and printed all elements. This demonstrated how dynamic memory allocation works in linked lists.

TASK-4

PROMPT: Generate a Python program to implement a hash table with insert, search, and delete operations using collision handling.

CODE:



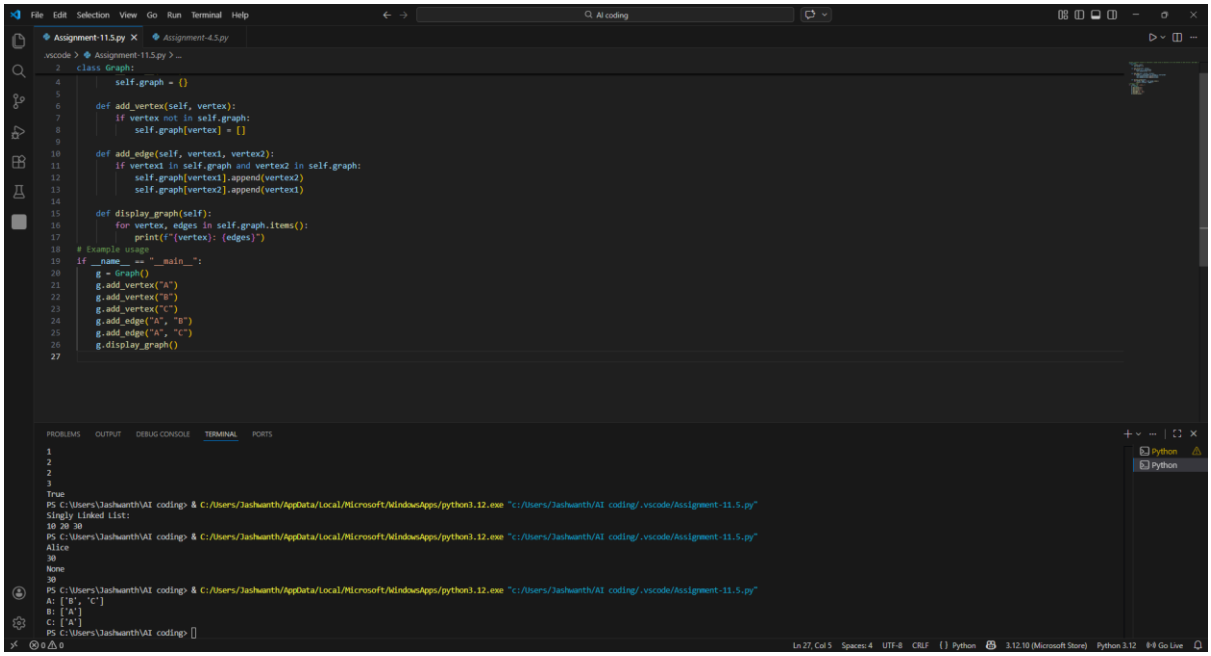
OBSERVATION:

The hash table successfully stored key-value pairs using a hashing function. Collision handling was implemented using chaining. The insert, search, and delete operations worked efficiently, showing how hash tables provide fast data retrieval.

TASK-5

PROMPT: Create a Python program to represent a graph using an adjacency list with methods to add vertices, add edges, and display the graph.

CODE:



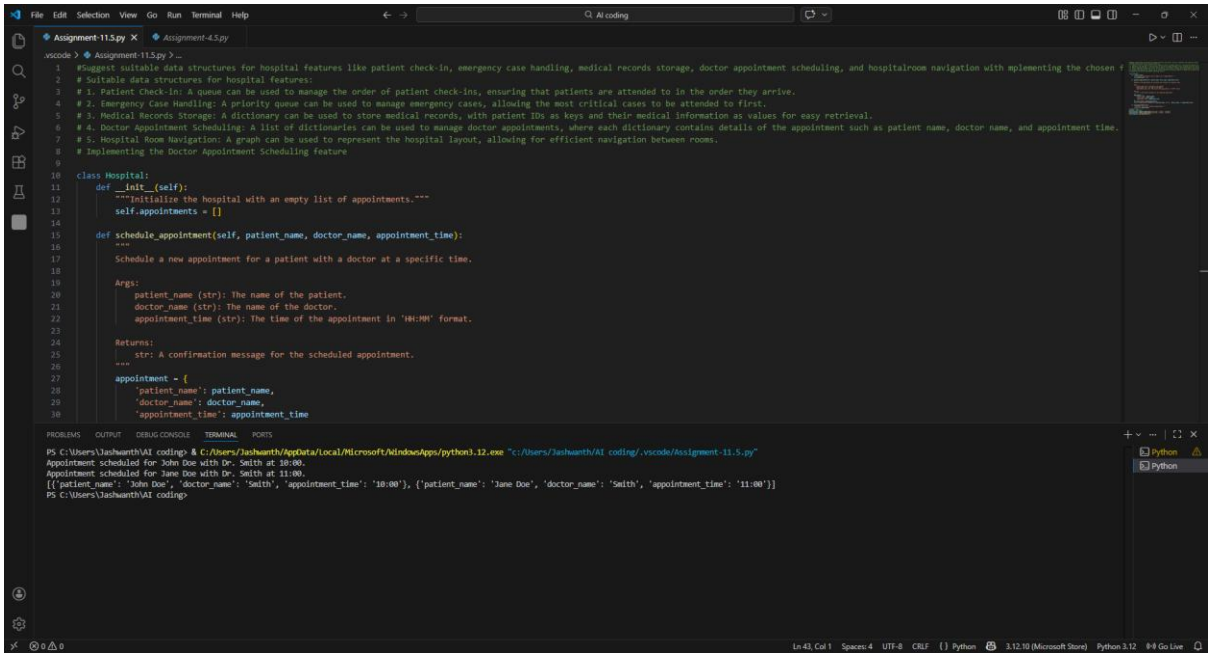
OBSERVATION:

The graph was represented using an adjacency list. Vertices and edges were added correctly, and the graph structure was displayed properly. This showed how graphs can represent relationships or connections between different nodes.

TASK-6

PROMPT: Suggest suitable data structures for hospital features like patient check-in, emergency handling, medical records, appointment scheduling, and room navigation with justification.

CODE:



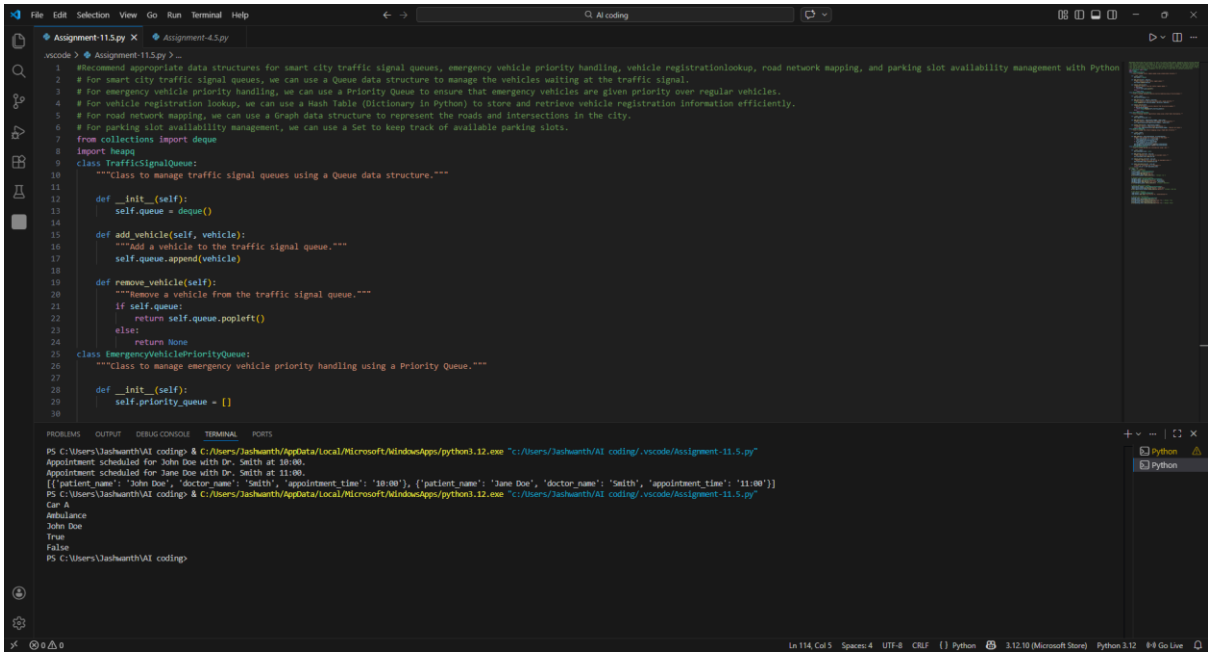
OBSERVATION:

Different data structures were selected for different hospital operations. For example, queues can manage patient check-ins, priority queues can handle emergency cases, dictionaries can store medical records, and graphs can represent hospital navigation. The appointment scheduling feature worked correctly.

TASK-7

PROMPT: Recommend appropriate data structures for traffic signal queues, emergency vehicle priority, vehicle lookup, road mapping, and parking management.

CODE:



OBSERVATION:

Appropriate data structures were used for traffic management features. Queues handled traffic signals, priority queues managed emergency vehicles, dictionaries stored vehicle registration data, graphs represented road networks, and sets tracked parking availability. The implementation demonstrated efficient handling of real-world traffic scenarios.

TASK-8

PROMPT: Choose suitable data structures for shopping cart management, order processing, top-selling products tracking, product search, and delivery route planning.

CODE:

OBSERVATION:

Different data structures were used to manage e-commerce operations. Dictionaries stored cart items and orders, sorting was used to find top-selling products, and search functionality helped locate products quickly. The system demonstrated how data structures help manage large-scale online shopping operations efficiently.